2.5 Day   9 – 5 IST

2 Coffee breaks < AM – 11:00 / 10:30 AM ; PM – 3:00 PM / 2:30 PM > 15 mins

Lunch Break – ~~12:30 PM / 1 HR~~ (+1) (+2) 1:00 PM IST (1 HR)

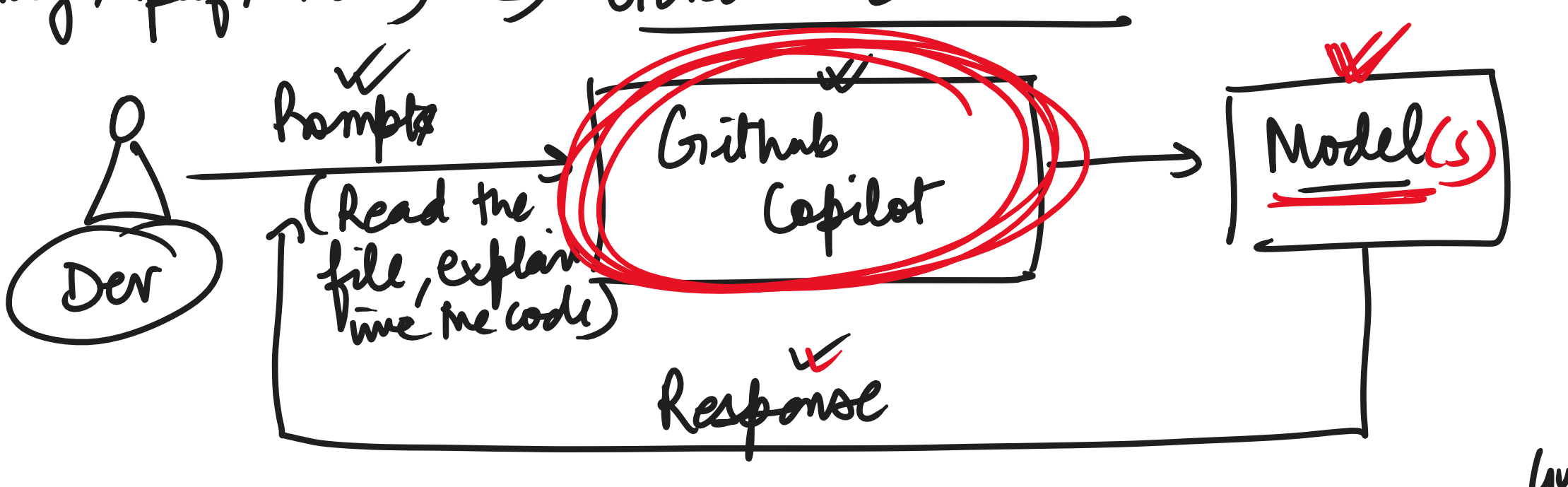
Yes = (+1)
No = +2

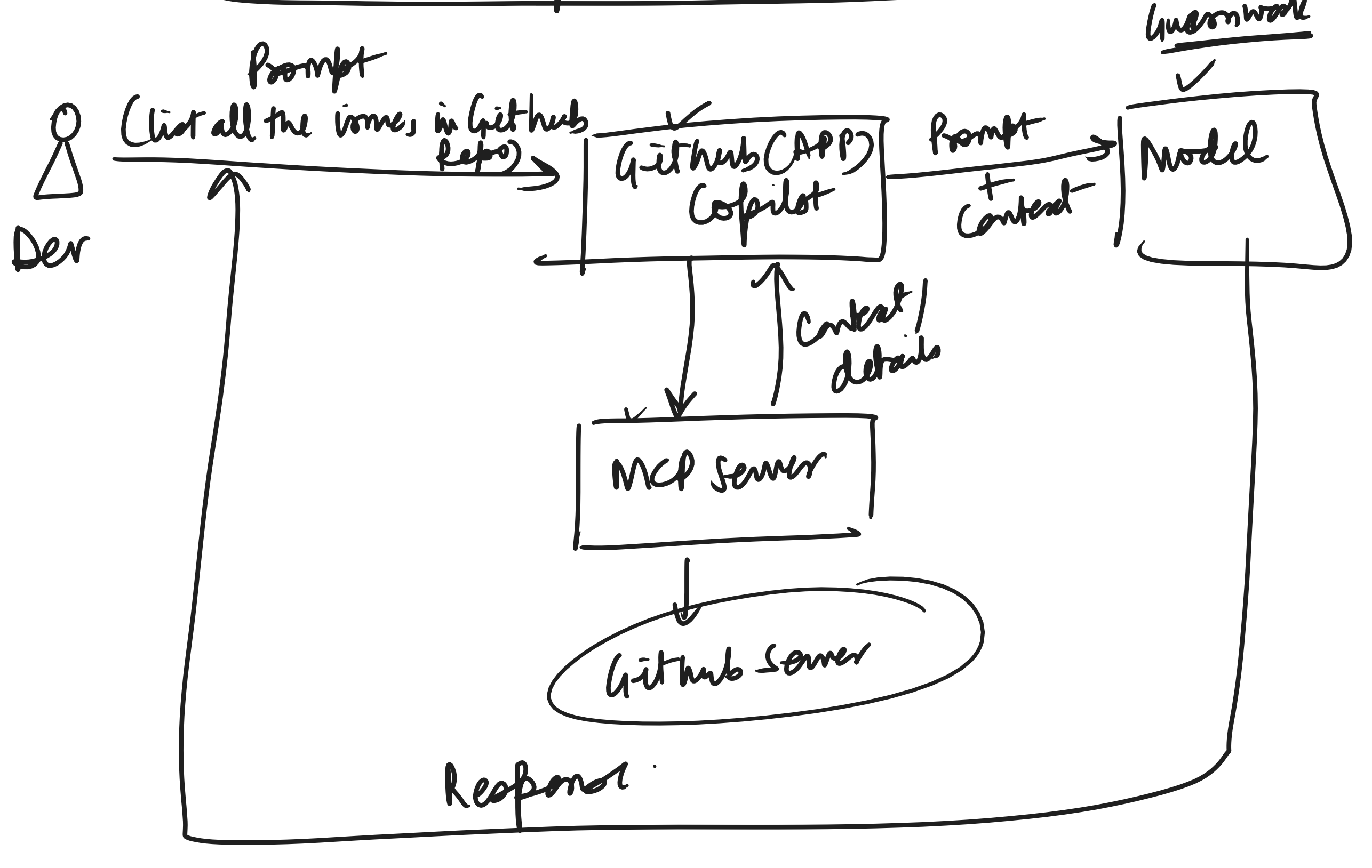
"Github Copilot ? "AI Coding Assistant"

JR. Dev → New to ORG / Domain / Project
→ understand the Program / write
→ Do whatever is explain in details

GenAI Application → Code Generation
↳ Powered By FM | LLM

FM/LLM are pretrained models which are trained on Internet scale of Data (Raw Data → JSON, code, text, image, pdf, video) ⇒ General Domain





8 → input / Prompt + Context (file + MCP + Proj struc)

Model → output / Response

Tokens

Input + output ⇒ Model

Total # of Tokens = credits

① Inline suggestions → # comments, Typing your code

② Inline chat = Ctrl + I → windows
cmd + i → Mac
↳ Accept → Tab key
↳ Cancel → Esc key

Chat mode

③ Inline chat / Copilot Chat → Agent Mode.

PROMPT Engineering

Context ⇒ Existing code, file structure, interfaces, examples
Rules: language, style, lib, performance & security rules
Target output ⇒ What exactly we want produced: a function, tests, refactor, explanation, debugging

Mental Model = Copilot is "context + Rules + output"

"Golden Prompt Template"

① Goal = what you want
② Input/outputs = func signature, data shape
③ Rules: Must / Must not
④ EdgeCases: nulls, empty array, invalid input
⑤ Examples: one / two
⑥ Deliverables: Code + Tests + Explanation

BEST Practices

① Give "Copilot" "THE WORLD IT LIVES within"
  ⓐ the file it has to edit
  ⓑ Nearby Types / Interfaces
  ⓒ sample data
  ⓑ existing patterns in Repo
② Ask for incremental output, not a whole system
  ① generate types + func signature
  ② Implement the core logic
  ③ Add ~~ta~~ validation
    Tests + Refactor
③ Control style & safety

Debug-style Prompting

Framework ⇒ observe → Hypothesize → Prove → Fix → Prevent

observe ⇒ error message + where it + minimal code
Hypothesize ⇒ Ask for #(2,3) likely cases
Prove ⇒ ask for checks / logs / tests to confirm
* Fix ⇒ apply minimal change
Prevent ⇒ add regression test + guardrails.

Prompts (Debug):
① Explain this error like I'm a junior dev, then propose the smallest fix.
② Show me 3 ranked by likelihood, then give steps to validate each
③ Add instrumentation logs (no secrets) to isolate where it fails.

Core Copilot use cases

① Code Generation (functions, classes, API)
② Refactoring & modernization
③ Test Generation
④ Documentation (docstrings, Readme.md)
⑤ Asking Qs about the existing code.

Intent → Prompt + Context → Review/validate

API Generation, "Copilot" may:
① Skip validation
② use inconsistent status codes
③ forget edge cases
④ mix patterns from different Express styles